

Unsupervised Learning Notes

Contents

Introduction to unsupervised learning	2
Idea of unsupervised learning	2
Readings	3
lab	3
Distance and similarity	3
Feature scaling and normalization	3
Distance metrics	4
Curse of dimensionality	6
Readings	6
lab	6
Clustering methods	6
Intro to clustering	6
Partitioning methods: K-means and k-medoids clustering	7
Hierarchical Methods	10
Density-based spatial clustering of applications with noise (DBSCAN)	12
Comparison of k-means, hierarchical, and DBSCAN methods	14
Assessing cluster quality	14
Readings	16
lab	16
Dimension reduction	16
Singular value decomposition (SVD)	17
Principal Component Analysis (PCA)	22
t-SNE (t-Distributed stochastic neighbor embedding)	25
UMAP (Uniform Manifold Approximation and Projection)	28

Introduction to unsupervised learning

Idea of unsupervised learning

1. Slideshow: What is AI, ML, data science, statistics, etc? How does it all fit? Cool diagrams.
2. Supervised learning vs unsupervised learning:

(a) Supervised learning:

- Given labeled data, predict a target. Example is predicting a student's GPA from student data.
- More formally, labeled training data is (X, y) for n samples with relationship

$$y = f(X) + \epsilon.$$

Goal is to approximate f via $\hat{f}$. Call supervised because we know answers from our training data.

- Basic example of linear regression with $p = 1$ for student data. Assume f is linear. Graph with data and best fit line $\hat{f}$. Write general p variable case. Can be used for student intervention (early warning, effect of living on campus, attending class).
- Main goals are prediction and inference by understanding f (feature importance, model fit and reliability).
- Linear model is an assumption. Other models include decision trees, neural networks, support vector machine, and more.
- Well understood area, clear ways to assess quality of results. ISLR text key reference.

(b) Unsupervised learning:

- Unsupervised learning is a class of machine learning methods used to discover structure in data

$$X = \{x_1, x_2, \dots, x_n\}, \quad x_i \in \mathbb{R}^p \quad (\text{note vector notation})$$

without labeled outcomes. Instead of predicting a known target, the goal is to explore, summarize, and reveal patterns that are intrinsic to the data.

- Given unlabeled data X , find structure in the data. No prediction, no supervision. Learning by observation, not by example.
- What types of students are there given hours studied per week and GPA? What variable combinations belong together (academic, engagement)? Unusual students? Goal is to better understand data.
- Can be a stand alone analysis or can be used to compliment supervised learning.

3. Core tasks of unsupervised learning:

- (a) Clustering: Group similar observations in the same cluster. K-means, hierarchical, dbSCAN
- (b) Dimension reduction: Reduce noise and multicollinearity, data viz, large to smaller data, data understanding. PCA, t-SNE, UMAP, SVD. Google tensorflow embedding projector.
- (c) Anomaly detection: Learn distribution of data to quantify outlier probabilities.

4. Concrete examples of unsupervised learning:

(a) Customer segmentation: Clustering

- Walmart data on spend average, frequency, mode, product mix, app use.

- No label such as budget shopper or family provider. Want to discover segments rather than predetermine behavior.
- Each data point is a customer in high dimensions.
- Similarity means close distance. Scaling and choice of distance matters.
- Possible clusters: Weekly family stockup, single essentials, deal hunters. These are business driven interpretations.
- Actions: Store layout optimization, personal coupons, inventory planning, regional differences. Not aiming for individual predictions (as with supervised learning).
- Google: Walmart customer segmentation

(b) Spotify music genre: Dimension reduction

- Google: Spotify api dataset
- Many automatic features, how to tell what genre?
- Vectors are high dimension, but human perception is low dimension.
- Can we compress data into low-dimensions?
- Distance reflect song similarity.
- Are there distinct groups of genres or continuous flow?
- Goal is to make data more intelligible.

(c) Credit card fraud: Anomaly detection

- Each data point is a transaction (amount, time, source, location, recent freq, device).
- Millions of these per day, tiny amount are fraud.
- False positive is a problem.
- Does this deviate from expected? Normal behavior is dense regions, but some deviance can naturally occur.
- Distance metric determines deviation from normal.
- Per customer (card?) normalization.
- Action may be to identify fraud patterns to catch more.

5. Much more challenge than supervised learning. No simple goal. Results are subjective. Exploratory, descriptive, and hypothesis-generating rather than predictive.

Readings

1. ISLR 2.1.4, Ch12 thru 12.1
2. HOUL Ch1

Lab

1. EDA and data cleaning, DMCT Ch2 and Ch3

Distance and similarity

Feature scaling and normalization

1. Features on a bigger scale dominate distance calculations. Spend vs weekly visit count for customer segmentation. Illustrate dollars vs thousands for spend on Euclidean distance.

$$d_2(x, y) = \|x - y\|_2 = \sqrt{\sum (x_j - y_j)^2}$$

Many ways to scale to prevent this issue.

2. Standardization (z -score scaling):

- (a) Definition: Shift by mean and divide by standard deviation.

$$x'_i = \frac{x_i - \bar{x}}{s_x}$$

where s_x is the standard deviation of variable x . Remind of standard deviation calculation. Recall sample vs population notation. Draw distribution picture, two different shapes.

$$s_x = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}, \quad \bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

- (b) Results in mean 0, variance 1. Now on a standard normal distribution $N(0,1)$. Nice properties of a normal distribution for 68-95-99 percent confidence intervals. Preserves order and relative distribution.
- (c) Equal feature contribution, preserves relative difference.
- (d) Extensions include mean absolute deviation.

3. Min-max scaling:

- (a) Definition: Shift by min and divide by range.

$$x'_i = \frac{x_i - \min(x)}{\max(x) - \min(x)}$$

- (b) Guess what it does. Refer to mall data scaling age.
- (c) Result is in interval $[0, 1]$. Preserves order but compresses extremes.
- (d) Good for bounded features.

4. Most of the time scale, can always reverse scaling if you keep track of original stats. When scaling is NOT a good idea:

- (a) Feature units are meaningful, important, and want to keep for analysis.
- (b) Binary indicators (0/1), changes to discrete values but different values.
- (c) Counts with semantic meaning, such as a 1-10 satisfaction rating.
- (d) Ratios which are already normalized.

Distance metrics and similarity

1. Distance is a choice we make which encodes different notions of similar. Many options.

2. Key properties are required: Mathematically called a metric in a metric space.

- (a) Positivity: $d(x, y) \geq 0$ for all x, y and $d(x, y) = 0$ only if $y = x$.
- (b) Symmetry: $d(x, y) = d(y, x)$.
- (c) Triangle inequality: $d(x, z) \leq d(x, y) + d(y, z)$ for any x, y, z . Illustrate with pictures, x and z on the horizontal axis.

3. Common distance metrics:

(a) Euclidean distance (ℓ_2 norm): For points $x, y \in \mathbb{R}^d$,

$$d_2(x, y) = \|x - y\|_2 = \sqrt{\sum (x_i - y_i)^2}$$

- Geometry: Straight-line distance, rotation invariant, penalizes large feature deviations heavily.
- Assumes: Features are commensurate, spherical neighborhoods make sense
- Example: Customers close if very same spend pattern and volume. (same shop and spend)

(b) Manhattan distance (ℓ_1 norm):

$$d_1(x, y) = \|x - y\|_1 = \sum |x_i - y_i|$$

- Geometry: City block distance, diamond shaped contours, less sensitive to large feature deviations.
- Assumes: More robust to outliers.
- Example: Customers close if same spend pattern and volume, some diff tolerated. (similar with occasional deviation allowed)

(c) Supremum (ℓ_∞ norm):

$$d_\infty(x, y) = \max\{|x_i - y_i|\}$$

- Geometry: Largest difference only. Shapes are squares.
- Example: Customers close if same spend pattern and volume, one big difference is a problem.

(d) These are all generalized by Minkowski distance:

$$d(x, y) = \|x - y\|_r = \sqrt[r]{\sum (x_i - y_i)^r}$$

Overall, Euclidean is the favorite, but there are good reasons to use others. Show animation in Desmos of different unit circles $x^r + y^r = 1$. $r = \infty$ case is the limit which calc 1 shows supremum norm is found. Show in 2D.

(a) Similarities: Note not a distance, but sometimes makes more sense.

(b) Cosine similarity:

$$\text{sim}_{\cos}(x, y) = \frac{x \cdot y}{\|x\| \|y\|} = \cos(\theta)$$

- Geometry: measures cosine of angle between rather than distance, lives in $[-1, 1]$, same direction is close to 1, orth and opposite direction, ignores scale. Turns our ordering agrees with Euclidean distance.
- Assumes: Pattern rather than intensity.
- Example: Customers close if same spend pattern but different spend volumes. (shop same regardless of spend)

(c) Correlation: Sample Pearson correlation.

$$\text{sim}_{\text{cor}}(x, y) = r = \text{cor}(x, y) = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}$$

- Geometry: Measures linear relationship, $r \approx 1$ strong positive linear relationship, $r \approx -1$ strong negative, in between. Book picture vs Wikipedia picture.
- Same as cosine similarity if data is normalized.

Curse of dimensionality

1. In high dimensions, all points become almost equally far apart. Nearest points are almost same distance as farthest.
2. As dimension d increases,

$$\frac{\max d(x, y) - \min d(x, y)}{\min d(x, y)} \rightarrow 0$$

Readings

1. IDM Ch 2 Intro, 2.3.7, 2.4.1-2.4.5, 2.4.9-2.4.10 recommend all of Ch 2
2. DMCT Ch 2 Intro, 2.5.1, 2.3.1, 2.3.4, 2.3.7, 2.3.9 recommend all of Ch 2 except 2.6

Lab

Clustering methods

Main reference: DMCT chapter 9

Intro to clustering

1. What is cluster analysis?
 - (a) Clustering groups data into clusters so that objects within a cluster are more similar to each other than to objects in other clusters, according to a chosen notion of similarity.
 - (b) Clustering is an *ill-posed* problem: there is no single correct solution. Results depend on modeling assumptions, similarity definitions, and analysis goals. There is no universal notion of “true” clusters.
 - (c) Similarity (or dissimilarity) is typically defined through a distance or proximity measure.
 - (d) Different clustering methods reflect different assumptions about data structure (e.g., partitioning, hierarchical, density-based, grid-based). Some methods that optimize an explicit objective (k-means) while others identify structure without a global objective (hierarchical, DBSCAN).
 - (e) Clustering quality can be assessed in multiple ways, including internal criteria, external validation, stability, and interpretability.
 - (f) Ongoing research focuses on scalability, high-dimensional settings where distance metrics break, complex cluster shapes, and diverse data types (e.g., text, images).
2. Desiderata for clustering methods:
 - (a) Ability to handle different data types (numeric, categorical, mixed)
 - (b) Ability to detect non-spherical or non-convex clusters
 - (c) Robustness to noise and outliers
 - (d) Scalability to large datasets
 - (e) Ability to incorporate constraints or side information
 - (f) Results that are interpretable and actionable
3. Types of clustering methods

- (a) Partitioning methods (customer segmentation, geog segm)
 - Assume clusters are compact, well-separated, and cover all data points.
 - Partition n objects into k non-overlapping clusters.
 - Typically distance-based and solved via iterative optimization.
 - Sensitive to initialization, distance choice, and cluster shape assumptions.
- (b) Hierarchical methods (fish in Mississippi, species, genus, family, order, class, phylum, kingdom)
 - Assume nested cluster structure is meaningful.
 - Produce a hierarchy of clusters represented as a tree.
 - Agglomerative (bottom-up) or divisive (top-down) approaches.
 - Once a merge or split occurs, it cannot be undone.
- (c) Density-based methods (social data with natural crowding, maybe no fixed k or hierarchy)
 - Assume clusters correspond to regions of high data density separated by low-density regions.
 - Clusters are grown based on neighborhood density criteria.
 - Naturally identify outliers and allow arbitrary cluster shapes.
- (d) Key challenges and limitations:
 - Evaluation is inherently difficult due to lack of ground truth; metrics often encode the same assumptions as the algorithm.
 - Clustering is exploratory rather than confirmatory.
 - Domain knowledge plays a central role (feature selection, scaling, similarity choice, interpretation, clustering alg choice).
 - Clustering is not classification, causal inference, or discovery of objective real-world categories; results should not be over-interpreted.

Partitioning methods: K-means and k-medoids clustering

1. Big picture:

- (a) Given data $D = \{x_1, \dots, x_n\} \subset \mathbb{R}^p$, partition into k disjoint clusters $C_1, \dots, C_k$, $C_i \cap C_j = \emptyset$.
- (b) Each point belongs to exactly one cluster, and each cluster is summarized by a single representative (center).
- (c) Clustering defined via optimization of an objective function called within-cluster loss.

2. Key assumptions:

- (a) A meaningful distance $d(x, y)$ exists (eg Euclidean distance).
- (b) Clusters are compact and well-separated in the chosen metric.
- (c) Cluster centers summarize cluster geometry.
- (d) All data belong to some cluster (no noise model).
- (e) k is fixed and meaningful.

3. k -means: A centroid-based technique

- (a) Cluster center: centroid (mean) $c_i \in \mathbb{R}^p$.

(b) Objective function to minimize:

$$\min_{C_1, \dots, C_k} \sum_{i=1}^k \sum_{x_j \in C_i} d(x_j, c_i)^2, \quad c_i = \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j.$$

where the distance metric is Euclidean distance (almost always)

$$d(x, y) = \|x - y\| = \sqrt{(x_1 - y_1)^2 + \dots + (x_p - y_p)^2}.$$

The centroid represents a hypothetical average cluster member (customer).

(c) The within cluster variance is the sum of square errors:

$$WCSS = \sum_{i=1}^k \sum_{x_j \in C_i} d(x_j, c_i)^2$$

(d) Algorithm (Lloyd's algorithm):

- This optimization problem is computationally expensive (NP-hard), many ways to select k clusters of n data points. A basic iterative algorithm is used instead.
- Initialize $c_1, \dots, c_k$ as k random objects from D .
- Cluster assignment step: Belong to nearest centroid.

$$x_j \mapsto \arg \min_i \|x_j - c_i\|^2.$$

- Centroid update step: Average of cluster members.

$$c_i \leftarrow \frac{1}{|C_i|} \sum_{x_j \in C_i} x_j.$$

- Iterate until assignments stabilize.

- (e) Each step decreases the objective, converges to a local minimum, may not be global minimum.
- (f) Implies Voronoi partition of the feature space, boundaries between clusters where two centroids are equidistant, only depends on centroid not data distribution.
- (g) Implicit assumptions: spherical clusters, equal variance, Euclidean geometry.
- (h) Variations of k -means involved different distance metrics, smart centroid initialization, and centroid calculation strategies, k -modes for nominal data, groupings of data called microclusters.

4. k -medoids: A representative object-based technique

- (a) Motivation: k -means is sensitive to outliers when centroids (means) are calculated.
- (b) Cluster center: medoid $o_i \in x_1, \dots, x_n$. Centroid is now a data point. Also called a representative object.
- (c) Objective function to minimize: Note that medoids o must be data points, so computing new centers is not obvious.

$$\sum_{i=1}^k \sum_{x_j \in C_i} d(x_j, o_i)$$

Distance $d(\cdot, \cdot)$ need not be Euclidean. Note the lack of squared distance.

- (d) More robust to outliers (no averaging).
- (e) Example algorithm: Partitioning around medoids PAM
 - Random initial medoids. Assign cluster membership. Brute force check all updated medoids per assigned cluster to see which minimizes total distance error. Iterate.

$$WCTD = \sum_{i=1}^k \sum_{x_j \in C_i} d(x_j, o_i)$$

- Modification for large data, clustering large applications (CLARA) considers random samples of the dataset.
 - (f) Slower than k -means due to discrete optimization, especially for large n and large k .
5. k -means vs k -medoids (mathematical contrast)
- Continuous optimization (means) vs discrete optimization (medoids).
 - Squared Euclidean loss vs general metric loss.
 - Sensitive vs robust to outliers.
 - Fast gradient-like updates vs combinatorial search.

6. Practical issues:

- Objective function is typically non-convex $\Rightarrow$ multiple local minima. Repeat iterations can give different cluster solutions.
- Initialization matters (e.g., random vs k -means++ which chooses initial clusters far apart).
- Scaling changes the geometry of $|\cdot|$.
- Choice of k is a modeling decision, not a statistical estimate. If you don't know k , try many and compare results.

7. When partitioning methods work well

- Clusters roughly convex and isotropic (not stretched in a certain direction).
- Moderate dimension with meaningful distances, otherwise dimension reduction needed.
- Clear notion of “center.”
- Need fast baseline clustering.

8. When they fail

- Non-convex or nested clusters.
- Unequal cluster variances or densities.
- Strong outliers (especially k -means).
- High-dimensional distance concentration.

9. How to determine the right number of clusters? Many ways! Basic discussion here, more later.

- (a) Need k in advance, but the right number of clusters is often ambiguous.
- (b) Good to balance compressibility (simplifying data) with accuracy (cluster meaning and usability).
- (c) An easy approach considers plotting WCSS (within cluster sum of squares) for $k = 2, 3, 4, \dots, k$ and use the elbow method. Sharp bend says smaller improvements. Expect to decrease to 0. Subjective but something to go by. Better(?) ways!

10. Fun simulator: <https://clustering-visualizer.web.app/kmeans>

Hierarchical Methods

1. Big picture:

- (a) Hierarchical clustering builds a nested sequence of partitions.
- (b) Output is a tree (dendrogram), not a single clustering.
- (c) Clusters exist at multiple resolutions (choices of k).
- (d) No single "best" number of clusters is assumed a priori.
- (e) 2 main types: Agglomerative (bottom-up) and divisive (top-down)

2. Key assumptions:

- (a) Nested structure in the data is meaningful.
- (b) Pairwise dissimilarities capture relevant structure.
- (c) Early decisions (merges or splits) are trustworthy.
- (d) No noise model: all points participate in the hierarchy.

3. Agglomerative hierarchical clustering:

(a) Algorithm: Bottom-up procedure.

- Start with n singleton clusters $\{x_1\}, \{x_2\}, \dots, \{x_n\}$. Compute a pairwise distance matrix.
- Iterative merge the two closest clusters. The dissimilarity between these two clusters indicates the height in the dendrogram where the fusion is places.
- Computed updated pairwise inter-cluster dissimilarities among the $n - 1$ remaining clusters. Repeat merging.
- Merge low in dendrogram means joined clusters were similar. High merge means clusters were dissimilar (separation).

(b) Cluster-cluster distance (linkage)

- Requires a linkage function $D(C_a, C_b)$ for clusters C_a, C_b .
- Common choices:

$$\text{Single / Minimum / Nearest neighbor: } D(C_a, C_b) = \min_{x \in C_a, y \in C_b} d(x, y)$$

Allows long, thin, winding, non-convex clusters. Connectivity matters, global compactness does not.

$$\text{Complete / Maximum / Farthest neighbor: } D(C_a, C_b) = \max_{x \in C_a, y \in C_b} d(x, y)$$

Produces compact, spherical clusters and penalizes irregular shapes. Sensitive to outliers.

$$\text{Average: } D(C_a, C_b) = \frac{1}{|C_a||C_b|} \sum_{x \in C_a} \sum_{y \in C_b} d(x, y)$$

$$\text{Mean / Centroid: } D(C_a, C_b) = |m_a - m_b|$$

Average and mean are compromise between single and complete linkage.

- (c) Linkage choice encodes shape assumptions. Distance metric also important per above discussion (what similar should mean in application).

4. Ward's method (variance-based linkage)

- Merge clusters that minimally increase total within-cluster variance.
- Objective interpretation:

$$\begin{aligned}
W(C_i, C_j) &= WCSS(C_i \cup C_j) - WCSS(C_i) - WCSS(C_j) \\
&= \sum_{\vec{x} \in C_i \cup C_j} d(\vec{x}, \vec{c}_{ij})^2 - \sum_{\vec{x} \in C_i} d(\vec{x}, \vec{c}_i)^2 - \sum_{\vec{x} \in C_j} d(\vec{x}, \vec{c}_j)^2 = \frac{|C_i||C_j|}{|C_i| + |C_j|} \|\vec{c}_i - \vec{c}_j\|^2
\end{aligned}$$

where c denotes the cluster centroid as in k-means. Note only for Euclidean distance for last step.

- Closely related to k-means objective, though indirectly. Best of both worlds in a way.
- Favors compact, spherical clusters.

5. Dendrogram: Illustrate example with basic distance measures.

- Tree structure encoding merge order and merge distances.
- Vertical height = dissimilarity at which merge occurs. Note near in the horizontal direction does not mean points/clusters are near. Good discussions in ISLR,
- Cutting the tree at height h induces a partition.
- Different cuts correspond to different k .

6. How to decide cut?

- Unlike k-mean, HCA does not optimize a cost function such as WCSS. Other approaches are needed.
- Dendrogram viz is key.
- Elbow method of linkage distance vs merge step (A sharp increase in linkage distance = merging dissimilar clusters, cut just before the big jump)
- WCSS works for Ward's linkage because it is inherently close to k -means.
- Cophenetic Correlation Coefficient (CCC)
 - The cophenetic correlation coefficient (CCC) is specific to hierarchical clustering and is often overlooked.
 - Idea: How well does the dendrogram preserve the original pairwise distances?

$$CCC = \text{corr}(d_{ij}, \hat{d}_{ij})$$

where d_{ij} is the original distance between points i and j , and $\hat{d}_{ij}$ is the cophenetic distance, defined as the height at which points i and j merge in the dendrogram.

- Interpretation: Values close to 1 indicate that the hierarchical clustering preserves pairwise distances well. Low values indicate that the dendrogram substantially distorts the geometry of the data.
- Uses: Comparing linkage methods (single, complete, average, Ward), and choosing an appropriate distance metric.

7. Divisive hierarchical clustering

- Top-down approach.
- Start with all points in one cluster.
- Recursively split clusters.

- Less common due to computational cost.
- Conceptually closer to repeated partitioning.

8. When hierarchical clustering works well

- Data have meaningful nested or multi-scale structure.
- Moderate sample size.
- Interest in relationships between clusters, not just assignments.

9. When it fails

- Large datasets (computational and memory cost).
- Strong noise or chaining effects (single linkage).
- Early incorrect merges propagate upward.
- Noisy distance measurements.

Density-based clustering: DBSCAN

1. Big picture:

- (a) DBSCAN = Density-based spatial clustering of applications with noise
- (b) Clusters are defined as regions of high point density separated by regions of low density.
- (c) Does not impose global geometry (no centroids, no partition).
- (d) Explicitly allows noise and outliers.
- (e) Number of clusters is determined by the data, not fixed in advanced.

2. Key assumptions:

- (a) A meaningful distance metric exists.
- (b) Clusters correspond to dense regions in the metric space.
- (c) Density is approximately homogeneous within the clusters.
- (d) Low-density regions separate clusters.

3. Parameters:

- (a) $\varepsilon > 0$ (radius parameter).
- (b) $\text{minPts} \in \mathbb{N}$ (minimum number of neighbors)

4. Neighborhood definition:

- (a) ε -neighborhood of a point x :

$$N_\varepsilon(x) = \{y : d(x, y) \leq \varepsilon\}$$

- (b) $|N_\varepsilon(x)|$ gives the density of point x , including the point itself in this count.

5. Point types:

- (a) Core point: At least minPts within a neighborhood of that point. Point A in diagram if $\text{MinPts} \geq 7$.

$$|N_\varepsilon(x)| \geq \text{minPts}$$

- (b) Border point: Not core, but falls in the neighborhood of a point (within the border), or possibly many points. Point B in diagram.

$$|N_\varepsilon(x)| < \text{minPts}, \quad \text{but } x \in N_\varepsilon(y) \text{ for some core point } y$$

- (c) Noise point: x is neither core nor border. Point C in diagram.

6. Density reachability:

- (a) Directly density-reachable:

$$y \in N_\varepsilon(x), \quad x \text{ is a core point}$$

- (b) Density-reachable: chain of directly density-reachable points.
- (c) Density-connected: two points reachable from a common core point.

7. Cluster definition:

- (a) A cluster is a maximal set of density-connected points.
- (b) Noise points are not assigned to any cluster.

8. Algorithm (conceptual):

- (a) Identify all core points by checking neighborhood density of all possible points.
- (b) Grow clusters by connecting density-reachable points into the same cluster.
- (c) Label remaining points as noise or border.

9. Geometric consequences:

- (a) Can recover non-convex and arbitrarily shaped clusters.
- (b) No forced assignment of all points.
- (c) Cluster boundaries follow low-density regions.
- (d) No global partition space.

10. Comparison to partitioning methods

- (a) No centroids or objective function.
- (b) No Voronoi geometry.
- (c) k not specified.
- (d) Explicit noise handling.

11. Sensitivity and limitations

- (a) Choice of ε and minPts is critical.
- (b) Struggles with varying cluster densities.
- (c) Distance concentration in high dimensions degrades performance.
- (d) Sensitive to distance scaling.

12. When DBSCAN works well

- (a) Clusters separated by low-density regions.
- (b) Non-spherical, irregular shapes.

- (c) Presence of noise or outliers.
- (d) Low-to-moderate dimensional data.

13. When it fails

- (a) Clusters with significantly different densities.
- (b) High-dimensional data.
- (c) Data without clear density gaps.
- (d) Poorly chosen distance metric.

Comparison of k-means, hierarchical, and DBSCAN methods

1. Key idea: Clustering methods are not interchangeable algorithms. They encode fundamentally different notions of what a cluster is.
2. The question each method answers:
 - (a) Partitioning (k-means and k-modes): Given k , how should I divide all points to minimize within-cluster dissimilarity?
 - (b) Hierarchical clustering: How are points related across multiple scales of similarity?
 - (c) DBSCAN: Which points belong to the same dense region, and which points are noise?
3. Mathematics of the machines:
 - (a) Partitioning:
 - Force the data into k compact clusters by minimizing within-cluster loss
 - Imposes Voronoi partition where cluster boundaries are hyperplanes
 - Parameters are k , distance metric, scaling, initialization
 - Fails for non-convex shapes, unequal variance, noise
 - (b) Hierarchical:
 - Reveal nested similarity structure through greedy merges or splits
 - No global geometric partition and shape depends on linkage choice
 - Parameters are distance metric, linkage, cut height
 - Fails for noise, chaining, large n
 - (c) DBSCAN:
 - Identify dense regions separated by low-density gaps and label the rest as noise
 - No partition of space and geometry adapts to data distribution
 - Parameters are ε , minPts, distance metric
 - Fails for varying densities, high dimension

Assessing cluster quality

1. Fundamental difficulty:
 - (a) Clustering is unsupervised: typically no ground truth labels, may not even be clusters in the first place.
 - (b) "Good clustering" is not uniquely defined. Many metrics.
 - (c) Evaluation criteria often encodes the same assumptions as the algorithm, can be deceptive.

- (d) Different metrics may rank the same clustering very differently.
- (e) Validation is about usefulness and stability, not correctness.

2. Three perspectives on validation:

- (a) Internal validation: use only the data and clustering structure
- (b) External validation: compare to known labels (when available)
- (c) Relative validation: compare multiple clusterings to each other

3. Internal validation (geometry-based):

- (a) Measures compactness (within-cluster similarity) and separation (between-cluster dissimilarity).
- (b) Common quantities we've seen, algorithm specific:

$$WCSS = \sum_{k=1}^K \sum_{i \in C_k} \|x_i - \mu_k\|^2 \quad (\text{within-cluster dispersion, built into kmeans})$$

$$CCC = cor(d_{ij}, \hat{d}_{ij}) \quad (\text{cophenetic correlation, evaluates dendrogram vs data distance, hierarchical})$$

- (c) Calinski-Harabasz (CH) index: measures how well-separated clusters are relative to how compact they are.

$$CH = \frac{SSB/(K-1)}{SSW/(n-K)} = \frac{\sum_k n_k d_2(\mu_k, \mu)^2 / (k-1)}{\sum_k \sum d_2(x_i, \mu_k)^2 / (n-k)}$$

for k the number of clusters and n the sample size.

- Between-cluster variance $\rightarrow$ clusters should be far apart
 - Within-cluster variance $\rightarrow$ clusters should be tight
 - Higher CH the better
- (d) Silhouette score: measures how well each point fits within its assigned cluster compared to other clusters.
- For point i :

$$a(i) = \text{average distance to points in same cluster}$$

$$b(i) = \min_{k \neq c(i)} \text{average distance to cluster } k$$

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}} \in [-1, 1]$$

- $s \approx 1$: well-clustered
- $s \approx 0$: ambiguous
- $s < 0$: likely misclustered
- Assumes distance-based, compact clusters, similar cluster density. Will penalize DB-SCAN
- Silhouette is an internal validation metric like W and B , but it evaluates clustering locally at the point level rather than globally at the centroid level. Can average per cluster to get cluster metrics. Also average all for full clustering.

4. Stability-based validation:

- (a) Idea: Good clusters should be reproducible under small perturbations.
- (b) Methods: Compare the data cluster solution to a new solution from...
 - Subsampling the data
 - Adding noise
 - Bootstrap resampling

(c) Compare cluster assignments across perturbations.

- Jaccard index

$$J = \frac{|C_j \cap \hat{C}_j|}{|C_j \cup \hat{C}_j|}$$

where $\hat{C}_j$ denotes the sampled / noise / bootstrap

- Also adjusted Rand index (pairs of points in same cluster vs not adjusted for randomness).

(d) Unstable clustering suggests:

- No strong structure
- K too large
- Model overly sensitive to noise

(e) Stability focuses on reliability rather than geometry.

Readings

1. DMCT Ch10, cluster analysis basic concepts and methods
2. DMCT 10.1, intro to cluster analysis
3. DMCT 10.2, partition methods (k-means and k-medoids)
4. ISL 12.4.1, k-means clustering
5. DMCT 10.3, hierarchical methods
6. ISL 12.4.2, hierarchical clustering
7. DMCT 10.4.1, DBSCAN
8. ISL 12.4.3, practical issues in clustering
- 9.

Lab

Dimension reduction

Key motivations for dimension reduction:

1. Fight the curse of dimensionality
2. Discover latent structure within data (key combinations of variables)
3. Remove noise dimensions from data
4. 2/3D visualization and model diagnostics

Techniques we will see:

1. Singular value decomposition (SVD): Foundation for many modern computational algorithms, key framework for dimension reduction capturing dominant geometric directions
2. Principal component analysis (PCA): Decompose data into most statistically descriptive factors, follows from SVD
3. t-distributed stochastic neighbor embedding (t-SNE): Nonlinear method focused on preserving local neighborhoods, cutting edge technique
4. Uniform manifold approximation and projection (UMAP): approximate manifold (surface) representation of data, cutting edge technique

Demos:

1. <https://timbaumann.info/svd-image-compression-demo/>
2. <https://projector.tensorflow.org/>
3. <https://www.tensorflow.org/datasets/catalog/mnist>
4. <https://www.tensorflow.org/text/tutorials/word2vec>

Singular value decomposition (SVD)

1. SVD idea:
 - (a) Determine a low-dimensional approximation of high-dimension data in terms of dominant patterns.
 - (b) It finds orthogonal directions in which the matrix stretches space, ordered by importance. Show transformation picture.
 - (c) Example of a matrix factorization which works on any matrix $X_{n \times m}$. Solid linear algebra conversation.
 - (d) Recap linear algebra and key factorizations:
 - Matrix mult
 - GE result LU
 - Eigen decomposition for a square matrix PDP^{-1}
 - Spectral theorem for symmetric matrices: Eigen decomposition becomes PDP^T , orthogonal eigenvectors, special case of SVD below.
 - QR decomposition, Q orthonormal columns, orthogonalize the columns
2. SVD formation:
 - (a) Data matrix $X = [\vec{x}_1 \dots \vec{x}_m]$, $\vec{x}_i \in \mathbb{R}^n$, though vectors could be complex valued. Typically $n \gg m$, tall skinny matrix.
 - (b) Factorization:

$$X_{n \times m} = U_{n \times n} \Sigma_{n \times m} V_{m \times m}^T$$

where U, V are unitary (orthonormal columns) containing left and right singular vectors (similar to eigenvectors)

$$U = [\vec{u}_1 \dots \vec{u}_n]_{n \times n}, \quad U^T U = U U^T = I$$

$$V = [\vec{v}_1 \dots \vec{v}_m]_{m \times m}$$

and Σ is a diagonal matrix of singular values (similar to eigenvalues)

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_m \\ \hline & & & 0 \end{bmatrix}$$

where

$$m = \text{rank}(X), \quad \sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_m > 0.$$

σ_i is ordered by importance.

- (c) Note that $X_{n \times m}$ has $n \gg m$, long rectangular matrix. Draw "shapes" of other matrices.
- (d) The SVD always exists and is unique.
- (e) Because of the zeros in Σ , not all of U is needed. This reduces to the so-called "economy" SVD.

$$X = U \Sigma V^T = [\hat{U} \quad \hat{U}^\perp] \begin{bmatrix} \hat{\Sigma} \\ 0 \end{bmatrix} V^T = \hat{U} \hat{\Sigma} V^T.$$

where $\hat{U}_{m \times m}, \hat{\Sigma}_{m \times m}$.

3. SVD for matrix approximation:

(a)

$$X = [\vec{x}_1 \vec{x}_2 \dots \vec{x}_m] = \hat{U} \hat{\Sigma} V^T = [\vec{u}_1 \vec{u}_2 \dots \vec{u}_m]_{n \times m} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_m \end{bmatrix}_{m \times m} \begin{bmatrix} \vec{v}_1^T \\ \vec{v}_2^T \\ \vdots \\ \vec{v}_m^T \end{bmatrix}_{m \times m}$$

Do the multiplication.

$$X = \sigma_1 \vec{u}_1 \vec{v}_1^T + \sigma_2 \vec{u}_2 \vec{v}_2^T + \cdots + \sigma_n \vec{u}_n \vec{v}_n^T$$

This is the sum of n rank 1 matrices (with unit columns, magnitude 1) ordered by contribution to matrix X .

- (b) Often truncate at rank r to get

$$X \approx \tilde{U} \tilde{\Sigma} \tilde{V}^T$$

noting that $\tilde{U}^T \tilde{U} = I_{r \times r}$ but $\tilde{U} \tilde{U}^T \neq I$.

- (c) The Eckard-Young theorem (1936) says this is the best possible rank r approximation of X .

$$\arg \min_{\tilde{X} \text{ st } \text{rank}(\tilde{X})=r} \|X - \tilde{X}\|_F = \tilde{U} \tilde{\Sigma} \tilde{V}^T$$

where the matrix Frobenius norm is

$$\|A\|_F = \sqrt{\sum (A_{ij})^2}$$

4. Intuition behind SVD: Dominant correlations.

- (a) Data matrix where $n \gg m$.

$$X = [\vec{x}_1 \ \vec{x}_2 \ \dots \ \vec{x}_m]_{n \times m}$$

- (b) Correlation matrix:

$$X^T X = \begin{bmatrix} \vec{x}_1^T \\ \vec{x}_2^T \\ \vdots \\ \vec{x}_m^T \end{bmatrix} [\vec{x}_1 \ \vec{x}_2 \ \dots \ \vec{x}_m] = \begin{bmatrix} \vec{x}_1^T \vec{x}_1 & \vec{x}_1^T \vec{x}_2 & \dots & \vec{x}_1^T \vec{x}_m \\ \vec{x}_2^T \vec{x}_1 & \vec{x}_2^T \vec{x}_2 & \dots & \vec{x}_2^T \vec{x}_m \\ \vdots & \vdots & \ddots & \vdots \\ \vec{x}_m^T \vec{x}_1 & \vec{x}_m^T \vec{x}_2 & \dots & \vec{x}_m^T \vec{x}_m \end{bmatrix} = [\vec{x}_i^T \vec{x}_j]_{m \times m}$$

This is a matrix of inner products of columns. Likewise XX^T is the matrix of inner products of rows.

- (c) Assuming $X = \hat{U} \hat{\Sigma} V^T$ and so $X^T = V \hat{\Sigma}^T \hat{U}^T$, then

$$X^T X = V \hat{\Sigma}^T \hat{U}^T \hat{U} \hat{\Sigma} V^T = V \hat{\Sigma}^2 V^T$$

and so

$$X^T X V = V \hat{\Sigma}^2$$

and V give the right eigenvectors with eigenvalues on the diagonal of $\hat{\Sigma}^2$. Likewise, for the left singular vectors:

$$X X^T = \hat{U} \hat{\Sigma} V^T V \hat{\Sigma}^T \hat{U}^T = \hat{U} \hat{\Sigma}^2 \hat{U}^T$$

and

$$X X^T \hat{U} = \hat{U} \hat{\Sigma}^2$$

so $\hat{U}$ gives the left eigenvectors of XX^T with the same eigenvalues $\hat{\Sigma}^2$.

- (d) Full derivation: **Short derivation of SVD:**

Let $X \in \mathbb{R}^{n \times m}$.

- i. Compute $X^T X$, which is symmetric and positive semidefinite:

$$X^T X \in \mathbb{R}^{m \times m}.$$

This gives the cleanest case for eigenstuff in all of linear algebra. Eigenvalues are all real and positive. Eigenvectors corresponding to distinct eigenvalues are orthogonal. Spectral theorem gives orthogonal diagonalization.

- ii. Find eigenvectors v_i and eigenvalues $\lambda_i \geq 0$ of $X^T X$:

$$X^T X v_i = \lambda_i v_i, \quad i = 1, \dots, r.$$

- iii. Define singular values:

$$\sigma_i = \sqrt{\lambda_i} \geq 0.$$

- iv. Compute left singular vectors:

$$u_i = \frac{X v_i}{\sigma_i} \in \mathbb{R}^n \quad \Rightarrow \quad X v_i = \sigma_i u_i.$$

- v. Assemble SVD:

$$X = U \Sigma V^T$$

where

$$\bullet U = [u_1, \dots, u_r] \in \mathbb{R}^{n \times r},$$

- $V = [v_1, \dots, v_r] \in \mathbb{R}^{m \times r}$,
 - $\Sigma = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$,
- and $r = \text{rank}(X)$.

Extend U and V to orthonormal bases for $\mathbb{R}^n$ and $\mathbb{R}^m$ to get a full SVD.

5. Geometry of the SVD: Unitary transforms

- (a) $X = U\Sigma V^T$. U is an orthonormal basis for the column space of X . V likewise for the row space of X .
- (b) Both U and V are unitary matrices. That is, $UU^T = U^T U = I_{n \times n}$, and $VV^T = V^T V = I_{m \times m}$. Unitary matrices preserve angles between and lengths resulting in a pure rotation.
- (c) X acts as

$$\text{rotation } (V^T) \rightarrow \text{stretch } (\Sigma) \rightarrow \text{rotation } (U).$$

- (d) X maps the unit sphere in $\mathbb{R}^m$ to an ellipsoid in $\mathbb{R}^n$.
- (e) Principal direction

$$Xv_i = \sigma_i u_i,$$

so v_i are orthonormal input directions, u_i are orthonormal output directions, and σ_i are the stretch factors.

- (f) Extremal property:

$$\sigma_1 = \max_{\|x\|=1} \|Xx\|,$$

i.e., σ_1 is the maximum stretching factor.

- (g) Image and discussion on Wikipedia.

6. Example: Find the singular value decomposition of

$$A = \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix}.$$

Solution:

Compute AA^T :

$$AA^T = \begin{bmatrix} 17 & 8 \\ 8 & 17 \end{bmatrix}.$$

Eigenvalues of AA^T :

$$\lambda_1 = 25, \quad \lambda_2 = 9$$

so the singular values are

$$\sigma_1 = 5, \quad \sigma_2 = 3.$$

Eigenvectors of AA^T (columns of U):

$$u_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad u_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Construct Σ :

$$\Sigma = \begin{bmatrix} 5 & 0 & 0 \\ 0 & 3 & 0 \end{bmatrix}.$$

Compute V from $v_i = \frac{1}{\sigma_i} A^T u_i$:

$$v_1 = \frac{1}{3} \begin{bmatrix} 2 \\ 2 \\ 1 \end{bmatrix}, \quad v_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \\ 0 \end{bmatrix}, \quad v_3 = \frac{1}{3} \begin{bmatrix} -1 \\ -1 \\ 2 \end{bmatrix}.$$

Then the SVD is

$$A = U \Sigma V^T, \quad U = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad V = [v_1 \ v_2 \ v_3].$$

7. Example: Find the singular value decomposition of

$$A = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix}.$$

Solution:

Step 1: Compute the smaller matrix AA^T (2×2):

$$AA^T = \begin{bmatrix} 3 & 2 \\ 2 & 3 \\ 2 & -2 \end{bmatrix} \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix} = \begin{bmatrix} 13 & 12 \\ 12 & 13 \end{bmatrix}.$$

Step 2: Find eigenvalues of AA^T :

$$\lambda_1 = 25, \quad \lambda_2 = 1$$

so the singular values are

$$\sigma_1 = 5, \quad \sigma_2 = 1.$$

Step 3: Eigenvectors of AA^T give the columns of U :

$$u_1 = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad u_2 = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix}.$$

Step 4: Compute V from $v_i = \frac{1}{\sigma_i} A^T u_i$:

$$v_1 = \frac{1}{5} \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix}^T \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \frac{1}{5\sqrt{2}} \begin{bmatrix} 5 \\ 5 \\ 0 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 \\ 1 \\ 0 \end{bmatrix},$$

$$v_2 = \frac{1}{1} \begin{bmatrix} 3 & 2 & 2 \\ 2 & 3 & -2 \end{bmatrix}^T \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \end{bmatrix} = \frac{1}{\sqrt{2}} \begin{bmatrix} -1 \\ 1 \\ -2 \end{bmatrix}.$$

Step 5: Construct Σ (3×2):

$$\Sigma = \begin{bmatrix} 5 & 0 \\ 0 & 1 \\ 0 & 0 \end{bmatrix}.$$

Step 6: Assemble the SVD:

$$A = U \Sigma V^T, \quad U = \frac{1}{\sqrt{2}} \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad V = [v_1 \ v_2 \ v_3].$$

Principal Component Analysis (PCA)

1. Data setup.

Let

$$X = [\vec{x}_1 \ \vec{x}_2 \ \dots \ \vec{x}_n] \in \mathbb{R}^{m \times n}, \quad \vec{x}_i \in \mathbb{R}^m.$$

Each column represents one sample and each sample has m features.

Our goal is to reduce the dimension of each $\vec{x}_i$ from m to k where $k < m$.

Reducing dimension is useful because it mitigates the curse of dimensionality, captures dominant patterns in the data for downstream analysis, and allows for meaningful visualization in low dimensions.

2. Geometric motivation for PCA.

In many datasets, the cloud of points in $\mathbb{R}^m$ appears to have one dominant direction of spread. In two dimensions, this often looks like an elongated ellipse. The variance is much larger in one direction than in others. PCA formalizes this idea by identifying directions of maximal variance.

There is a strong connection between PCA and the singular value decomposition.

3. Recall the definition of covariance.

For two features f_j and f_k ,

$$\text{Cov}(f_j, f_k) = \frac{1}{n-1} \sum_{i=1}^n (f_j^{(i)} - \bar{f}_j)(f_k^{(i)} - \bar{f}_k).$$

As a special case,

$$\text{Cov}(X, X) = \text{Var}(X) = \frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2.$$

If $\text{Cov}(X, Y) > 0$, the variables tend to increase together. If $\text{Cov}(X, Y) < 0$, they move in opposite directions. If $\text{Cov}(X, Y) = 0$, there is no linear association.

4. Centering the data.

Before performing PCA, we center the data.

Define the sample mean

$$\bar{\vec{x}} = \frac{1}{n} \sum_{i=1}^n \vec{x}_i.$$

Define centered samples

$$\hat{\vec{x}}_i = \vec{x}_i - \bar{\vec{x}}.$$

Collecting these gives

$$\hat{X} = [\hat{\vec{x}}_1 \ \dots \ \hat{\vec{x}}_n].$$

From this point forward, we assume the data has been centered and simply write $X = \hat{X}$.

Centering ensures that the origin represents the mean of the dataset and simplifies the covariance formulas.

5. The sample covariance matrix.

The sample covariance matrix is defined by

$$C_X = \frac{1}{n-1} X X^T.$$

Expanding this expression,

$$C_X = \frac{1}{n-1} [\vec{x}_1 \ \dots \ \vec{x}_n] \begin{bmatrix} \vec{x}_1^T \\ \vdots \\ \vec{x}_n^T \end{bmatrix}.$$

Carrying out the multiplication,

$$C_X = \frac{1}{n-1} \begin{bmatrix} \vec{x}_1 \vec{x}_1^T & \dots & \vec{x}_1 \vec{x}_n^T \\ \vdots & \ddots & \vdots \\ \vec{x}_n \vec{x}_1^T & \dots & \vec{x}_n \vec{x}_n^T \end{bmatrix}.$$

The diagonal entries represent variances of the individual features. The off-diagonal entries represent covariances between features.

The total variance in the dataset is given by the trace:

$$\text{trace}(C_X) = \frac{1}{n-1} \sum_{i=1}^n \vec{x}_i^T \vec{x}_i.$$

Thus the trace of the covariance matrix equals the sum of the variances.

6. Example: house size and price.

Suppose the two features are house size and price. Then

$$C_X = \frac{1}{n-1} \begin{bmatrix} \text{size variance} & \text{covariance} \\ \text{covariance} & \text{price variance} \end{bmatrix}.$$

The trace becomes

$$\text{trace}(C_X) = \frac{1}{n-1} (\text{size variance} + \text{price variance}).$$

This equals the total variance in the data.

7. PCA factorization.

We now seek an orthonormal basis that removes correlation and orders directions by variance.

Write

$$X = UY, \quad \text{where } Y = U^T X.$$

Here

$$U = [\vec{u}_1 \ \vec{u}_2 \ \dots \ \vec{u}_m]$$

has orthonormal columns, and

$$Y = \begin{bmatrix} \vec{y}_1^T \\ \vec{y}_2^T \\ \vdots \\ \vec{y}_m^T \end{bmatrix}$$

contains the transformed coordinates.

The columns of U define new orthonormal directions in feature space. The rows of Y represent the data expressed in those new coordinates.

The components in Y are uncorrelated and are ordered in decreasing variance.

8. Connection to covariance diagonalization.

Using $X = PY$, we compute

$$C_X = \frac{1}{n-1} X X^T = \frac{1}{n-1} (PY)(PY)^T = P \left(\frac{1}{n-1} Y Y^T \right) P^T.$$

Thus,

$$C_Y = P^T C_X P.$$

If the transformed features in Y are uncorrelated, then C_Y is diagonal.

Therefore PCA finds an orthonormal matrix that diagonalizes the covariance matrix and orders the resulting components by decreasing variance.

9. Eigenvalue characterization of principal components.

Because C_X is symmetric,

$$C_X = \frac{1}{n-1} X X^T$$

is a real symmetric matrix. Therefore it admits an eigenvalue decomposition

$$C_X = U \Lambda U^T,$$

where the columns of U are orthonormal eigenvectors and

$$\Lambda = \text{diag}(\lambda_1, \lambda_2, \dots, \lambda_m)$$

contains the eigenvalues.

Using the change of variables $Y = U^T X$, we compute the covariance of the transformed data:

$$C_Y = \frac{1}{n-1} Y Y^T = \frac{1}{n-1} (U^T X)(U^T X)^T = U^T \left(\frac{1}{n-1} X X^T \right) U = U^T C_X U.$$

Substituting the eigenvalue decomposition,

$$C_Y = U^T (U \Lambda U^T) U = \Lambda.$$

Thus the covariance matrix becomes diagonal in the eigenvector basis. The transformed features are uncorrelated, and the variances along the new directions are precisely the eigenvalues.

If we order the eigenvalues

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_m,$$

then the first principal component is the eigenvector corresponding to λ_1 , the second corresponds to λ_2 , and so on. These directions maximize variance subject to orthogonality constraints.

Connection to SVD.

If

$$X = U \Sigma V^T$$

is the singular value decomposition of the centered data matrix, then

$$C_X = \frac{1}{n-1} X X^T = \frac{1}{n-1} U \Sigma^2 U^T.$$

Therefore the eigenvectors of C_X are the left singular vectors of X , and the eigenvalues satisfy

$$\lambda_i = \frac{\sigma_i^2}{n-1}.$$

PCA can therefore be computed directly from the SVD of the centered data matrix.

t-SNE (t-Distributed stochastic neighbor embedding)

1. Motivation.

While PCA captures global linear patterns, many datasets contain complex nonlinear structure. t-distributed stochastic neighbor embedding (t-SNE) is a nonlinear method designed to preserve **local neighborhoods**, making it ideal for visualizing clusters in high-dimensional data in 2 or 3 dimensions.

2. Intuition.

t-SNE works by thinking: “Points that are close together in high dimensions should stay close in the low-dimensional embedding, and points far apart can spread out.”

It does this by:

- Measuring high-dimensional similarity with Gaussians (nearby points have higher probability).
- Mapping points to a low-dimensional space so that a similar probability distribution (using heavy-tailed Student t-distribution to prevent collapsing of points) is matched.
- Minimizing the mismatch between these distributions, so local clusters are preserved even if global distances are distorted.

In short: **t-SNE preserves local neighborhoods, revealing clusters and manifold structure visually.**

3. Prerequisites:

- (a) Gaussian distribution: In stat we denote a normal distribution as $X \sim N(\mu, \sigma)$ (or $Z \sim N(0, 1)$). Capture this curve as a function called a Gaussian.

$$f(x | \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right),$$

where μ is the mean and σ^2 is the variance.

Key properties:

- Symmetric about the mean μ .
- 68-95-99.7 rule: about 68% of values within 1σ , 95% within 2σ , 99.7% within 3σ .
- The Gaussian arises naturally in sums of independent random variables (Central Limit Theorem).

Why this matters for t-SNE:

- t-SNE models the *similarity between points* in high-dimensional space using conditional probabilities $p_{j|i}$.
- These probabilities are computed using a Gaussian centered at each point: nearby points get higher probability, distant points lower.
- The Gaussian's variance σ_i controls the effective neighborhood size per data point, tuned via *perplexity*.
- Thus, understanding Gaussians is essential to grasp how t-SNE measures "local closeness" before mapping to a lower dimension.
- Gaussian's are non-linear giving this feature to t-SNE.

(b) Student's t-distribution:

$$f(y | \nu) = \frac{\Gamma\left(\frac{\nu+1}{2}\right)}{\sqrt{\nu\pi} \Gamma\left(\frac{\nu}{2}\right)} \left(1 + \frac{y^2}{\nu}\right)^{-\frac{\nu+1}{2}},$$

where ν is the degrees of freedom. For $\nu = 1$ we have

$$f(y) = \frac{1}{\pi(1+y^2)}$$

which we will see later.

Key properties:

- Symmetric about 0, like a Gaussian, but with *heavier tails*.
- As $\nu \rightarrow \infty$, it approaches the standard Gaussian.
- Heavy tails allow distant points to be modeled with small, nonzero similarity in the low-dimensional space.

Why this matters for t-SNE:

- In the low-dimensional embedding, t-SNE uses a Student t-distribution (often $\nu = 1$, i.e., Cauchy) to compute similarities q_{ij} .
- Heavy tails prevent "crowding": otherwise, many moderately distant points in high dimensions would collapse near the center in 2D.
- This choice preserves *local neighborhoods* while allowing moderate distances to remain meaningful, improving cluster separation.

(c) Gradient descent:

Idea: Gradient descent is an iterative optimization method that moves variables in the direction that reduces a function's value, step by step. **General formula:**

$$\theta^{(t+1)} = \theta^{(t)} - \eta \nabla_{\theta} F(\theta^{(t)}),$$

Notation:

- $\theta^{(t)}$: current value of the variable(s) at iteration t (in t-SNE, the low-dimensional coordinates y_i).
- η : learning rate, a small positive number controlling step size.
- $F(\theta)$: the function we want to minimize (in t-SNE, the mismatch between high-dimensional and low-dimensional similarities).
- $\nabla_{\theta} F(\theta)$: gradient of F with respect to θ , giving the direction of steepest increase.

Process:

- Start with initial guesses for y_i .
- Compute the gradient $\nabla_{y_i} F$ for each point.
- Update each y_i using the formula above to reduce F .
- Repeat until changes are small or a maximum number of iterations is reached.

Why this matters for t-SNE:

- t-SNE uses gradient descent to iteratively adjust low-dimensional points so that local neighborhoods are preserved.
- Each update moves points closer to neighbors and pushes apart distant points, producing meaningful clusters.
- Understanding the formula clarifies the step-by-step “learning” process of the embedding.

4. t-SNE idea: Converts high-dimensional distances into **probabilities** representing similarities:

(a) High-dimensional probability distribution:

- Draw 2D picture and pretend it is high dimensional. Want a similarity measure between 2 points x_i and x_j . Keep in mind similarity should be symmetric ($\text{sim}(x_i, x_j) = \text{sim}(x_j, x_i)$). Assuming a normal distribution, we translate Euclidean distance to probability.
- The similarity between points x_i and x_j is modeled as a conditional probability $p_{j|i}$, proportional to a Gaussian centered at x_i :

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)},$$

where σ_i is chosen to match a fixed **perplexity** (a user-specified effective number of likely neighbors). Note that this definition has $p_{i|i} = 0$. Also similarities add to 1 via the fraction normalization.

- Symmetrize probabilities: Note that above conditional probability is not symmetric, so we average over dimension n to make it so.

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}.$$

(b) In the low-dimensional embedding (typically 2D), define a similar probability q_{ij} using a Student t-distribution with 1 degree of freedom (heavy-tailed), centered at y_i :

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq i} (1 + \|y_i - y_k\|^2)^{-1}}.$$

- (c) Now, all that remains is to find 2 dimensional vectors y_i so that these distributions are as similar as possible. We need a probability distribution similarity measure, then gradient descent will help us minimize.
- (d) Optimization: Kullback-Leibler (KL) divergence measures how one probability distribution Q differs from a reference distribution P .

$$\text{KL}(P\|Q) = \sum_i P(i) \log \frac{P(i)}{Q(i)}$$

- Comes from information theory to measure the expected information lost when approximating one probability distribution with another.

- $P(i)$ is the probability of outcome i under the “true” distribution.
- $Q(i)$ is the probability of outcome i under the approximating distribution.
- $\log \frac{P(i)}{Q(i)}$ measures how much “surprise” or information is gained by assuming Q instead of P for that outcome. Sign of log term signals Q over or under estimating.
- Multiplying by $P(i)$ gives the expected contribution from outcome i .
- KL divergence is always non-negative: $\text{KL}(P\|Q) \geq 0$.
- $\text{KL}(P\|Q) = 0$ only if $P = Q$ exactly.
- Intuition: it quantifies the “extra surprise” when using Q to approximate P .
- In t-SNE: P represents high-dimensional similarities, Q represents low-dimensional similarities. Minimizing KL ensures the low-dimensional map preserves neighborhood structure.

5. Key properties.

- Preserves local structure: nearby points in high dimensions stay nearby in the embedding.
- Can reveal clusters and nonlinear manifolds that PCA may miss.
- Distances between well-separated clusters may be distorted (global geometry not preserved).
- Sensitive to **perplexity** and learning rate.

6. Gradient descent update:

t-SNE finds the low-dimensional coordinates y_i by iteratively updating them to reduce the mismatch between p_{ij} and q_{ij} . The gradient descent step is

$$y_i^{(t+1)} = y_i^{(t)} - \eta \frac{\partial \text{KL}(P\|Q)}{\partial y_i},$$

where

- $y_i^{(t)}$ is the coordinate of point i at iteration t ,
- η is the learning rate,
- $\frac{\partial \text{KL}(P\|Q)}{\partial y_i}$ is the gradient of the KL divergence with respect to y_i , indicating the direction in which to move y_i to reduce divergence.

7. Practical tips.

- Preprocess data with PCA to 30–50 dimensions to speed computation and reduce noise.
- Run multiple times to check for stability, as t-SNE is stochastic.
- Use primarily for visualization rather than modeling, due to global distortion.

UMAP (Uniform Manifold Approximation and Projection)

UMAP is a nonlinear dimensionality reduction method based on manifold learning and topological ideas. It produces a low-dimensional embedding that preserves local neighborhood structure and some global structure of high-dimensional data. The algorithm can be described mathematically as follows:

1. Manifold assumption:

Suppose the data points

$$x_1, x_2, \dots, x_n \in \mathbb{R}^D$$

lie on a low-dimensional manifold M embedded in $\mathbb{R}^D$.

UMAP does not learn an explicit manifold equation. Instead, it approximates the manifold using the connectivity of nearby points, producing a weighted k -nearest neighbor graph:

$$G = (V, E),$$

where vertices correspond to data points and edge weights reflect the strength of local connectivity:

$$M \approx G$$

Whiteboard hint: draw a curved manifold replaced by a graph connecting nearby points.

2. Compute k -nearest neighbors:

- For each point x_i , compute distances to other points: $\text{dist}(x_i, x_j)$.
- Select the k nearest neighbors $\mathcal{N}_k(x_i)$.
- *Whiteboard hint:* circle neighbors of a point in 2D and label them.

3. Compute fuzzy membership probabilities:

- Assign a “fuzzy” membership to each neighbor:

$$\mu_{ij} = \exp\left(-\frac{\max(0, \text{dist}(x_i, x_j) - \rho_i)}{\sigma_i}\right)$$

where ρ_i is the distance to the nearest neighbor (ensuring each point is connected) and σ_i is set so that

$$\sum_{j \in \mathcal{N}_k(x_i)} \mu_{ij} \approx k$$

• Why fuzzy?

- Real neighborhoods are not binary; some neighbors are stronger than others.
- Fuzzy weights reflect the *strength of connectivity* between points.
- Smooths noisy data and allows continuous optimization via cross-entropy.
- *Whiteboard hint:* draw arrows of different thickness to neighbors.

4. Symmetrize the fuzzy graph:

UMAP produces directed fuzzy memberships μ_{ij} and μ_{ji} , which may differ due to asymmetric neighborhoods. To form an undirected edge, the weights are combined as

$$w_{ij} = \mu_{ij} + \mu_{ji} - \mu_{ij}\mu_{ji}.$$

Intuition:

- Combines both directions: either node can indicate a strong connection.
- Corrects for double-counting using the probabilistic “or” rule.
- Ensures symmetry ($w_{ij} = w_{ji}$) and keeps $0 \leq w_{ij} \leq 1$.

5. **Low-Dimensional Similarities:** In the low-dimensional embedding, UMAP defines a smooth similarity between points:

$$v_{ij} = \frac{1}{1 + a\|y_i - y_j\|^{2b}},$$

where $y_i, y_j \in \mathbb{R}^d$ are low-dimensional coordinates, and $a, b > 0$ control the decay.

Intuition:

- Nearby points: $v_{ij} \approx 1$, strongly connected.
- Distant points: $v_{ij} \approx 0$, weakly connected.
- Continuous weights allow gradient-based optimization to preserve high-D fuzzy connectivity.
- The low-dimensional similarity function

$$v(d) = \frac{1}{1 + ad^{2b}}$$

is fit *before embedding* to match the high-dimensional fuzzy graph. Using the high-D neighbor distances d_{ij} and weights μ_{ij} , a and b are chosen by minimizing

$$\sum_{i,j \in \text{neighbors}} (v(d_{ij}) - \mu_{ij})^2.$$

6. **Cross-entropy loss function:**

$$C = \sum_{i < j} \left[w_{ij} \log \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}} \right]$$

- Measures how well the low-dimensional similarities v_{ij} match the high-dimensional graph weights w_{ij} . - Intuition:

- High w_{ij} (strong connection in high-D) $\rightarrow$ large penalty if v_{ij} is small $\rightarrow$ points are pulled together.
- Low w_{ij} (weak or no connection) $\rightarrow$ large penalty if v_{ij} is large $\rightarrow$ points are pushed apart.
- Essentially, it is a “soft match”: connected points stay close, unconnected points separate.

- Gradient w.r.t y_i :

$$\frac{\partial C}{\partial y_i} = 2 \sum_j (v_{ij} - w_{ij}) \frac{\partial v_{ij}}{\partial y_i}$$

- Optimization via gradient descent moves points to reduce this mismatch.

7. **Optimization:** - Update coordinates via stochastic gradient descent:

$$y_i \leftarrow y_i - \eta \frac{\partial C}{\partial y_i}$$

- Local clusters form first, then global structure emerges. - *Whiteboard hint:* show iterative movement until clusters stabilize.

8. **Key hyperparameters:**

- `n_neighbors` — controls local vs global structure

- `min_dist` — minimum spacing between points; controls cluster tightness
- `metric` — distance metric in high-D space
- *Whiteboard hint:* draw same points with tight vs spread clusters for different values

	Feature	t-SNE	UMAP
9. Comparison with t-SNE:	High-D similarity	Pairwise probabilities p_{ij}	Fuzzy kNN graph w_{ij}
	Low-D similarity	Student t-distribution q_{ij}	Flexible kernel v_{ij}
	Loss function	KL divergence	Cross-entropy on fuzzy edges
	Global structure	Often distorted	Some global preserved
	Hyperparameters	Perplexity	<code>n_neighbors</code> , <code>min_dist</code>

10. **Intuition summary:** UMAP builds a fuzzy neighbor graph to represent local connectivity, then finds a low-dimensional embedding that preserves this structure: connected points stay close, unconnected points are pushed apart. By modeling only local neighborhoods, UMAP scales efficiently and captures both cluster structure and some global topology.